

PlantGuard AI

Automated Plant Disease Classification System

Using Convolutional Neural Networks

Project Report

Members

Muhammad Saeed (23f-Ai-11)

Muhammad Rafay Anwar (23f-Ai-17)

Muneeb-ur-Rehman (23f-Ai-19)

Muhammad Khuzaima Siddiqui (23f-Ai-29)

GitHub: github.com/MKS-24/Plant-Guard-AI

Scratch Model Demo: huggingface.co/spaces/Khuzaima123/plant-disease-classifier Scratch

Fine-Tune Model Demo: huggingface.co/spaces/Khuzaima123/plant-guard-ai

1. Introduction

Plant diseases are a major threat to global food security, causing crop losses estimated at 20-40% of global production annually. Early and accurate detection of leaf disease is critical to enable timely intervention and minimize yield loss.

This project, PlantGuard AI, delivers two complementary CNN-based disease classifiers — both deployed as live Flask web applications on Hugging Face Spaces via Docker:

- **Scratch Model:** A custom 3-block CNN trained from scratch (no pretrained weights) classifying 5 Tomato leaf disease classes. 89% accuracy, ~322K parameters, inference time <2 seconds.
- **Fine-Tuned Model (Plant Guard AI):** A pretrained model fine-tuned in two phases to classify 15 disease/healthy classes across Pepper Bell, Potato, and Tomato. 91.82% accuracy after fine-tuning, 2.42M trainable parameters, inference time <2 seconds.

2. Pipeline Architecture

Both models share the same end-to-end pipeline from image ingestion to classification output.

Stage	Description
1. Data Ingestion	Raw leaf images from PlantVillage dataset (JPG/PNG)
2. Preprocessing	Resize to target input size, normalize pixel values to [0, 1], one-hot encode labels
3. Augmentation	Random horizontal/vertical flips, rotation, zoom, brightness jitter (TensorFlow pipeline)
4. Conv Block 1	32 filters, 3x3 kernels — learns edges and basic color gradients
5. Conv Block 2	64 filters, 3x3 kernels — learns mid-level texture and lesion patterns
6. Conv Block 3	128 filters, 3x3 kernels — learns high-level disease signatures
7. Dense Head	GlobalAvgPool → Dense(256, ReLU) → Dropout → Dense(N, Softmax)
8. Output	N-class softmax probability vector; argmax gives predicted disease class

Each convolutional block follows: Conv2D → BatchNormalization → ReLU → Conv2D → BatchNormalization → ReLU → MaxPooling2D(2x2) → Dropout. GlobalAveragePooling2D replaces Flatten to reduce parameters and overfitting.

3. Dataset

3.1 Source

PlantVillage — a publicly available dataset of 38 plant-disease combinations across 14 crop species. Two subsets are used, one per model variant.

3.2 Scratch Model — 5 Disease Classes (from app.py CLASS_NAMES)

The Scratch model classifies 5 Tomato leaf conditions. These are the exact class names as defined in the deployed app.py:

Index	Class Name (exact)
0	Healthy
1	Early Blight
2	Late Blight
3	Leaf Mold
4	Septoria Leaf Spot

3.3 Fine-Tune Model — 15 Classes (from app.py and predict.py CLASS_NAMES)

The Fine-Tune model covers 15 classes across 3 crop species. These are the exact class names as defined in both app.py and predict.py:

Index	Class Name (exact)	Crop
0	Pepper_bell__Bacterial_spot	Pepper Bell
1	Pepper_bell__healthy	Pepper Bell
2	Potato__Early_blight	Potato
3	Potato__Late_blight	Potato
4	Potato__healthy	Potato
5	Tomato_Bacterial_spot	Tomato
6	Tomato_Early_blight	Tomato
7	Tomato_Late_blight	Tomato
8	Tomato_Leaf_Mold	Tomato
9	Tomato_Septoria_leaf_spot	Tomato
10	Tomato_Spider_mites_Two_spotted_spider_mite	Tomato
11	Tomato__Target_Spot	Tomato
12	Tomato_Tomato_YellowLeaf_Curl_Virus	Tomato
13	Tomato__Tomato_mosaic_virus	Tomato
14	Tomato_healthy	Tomato

3.4 Preprocessing

- **Resize** — Target size read dynamically from `model.input_shape[1]` and `model.input_shape[2]` at inference time.
- **Normalization** — pixel values divided by 255.0, mapping to [0.0, 1.0] (`img_array = img_resized.astype('float32') / 255.0`).
- **Label Encoding** — Class labels one-hot encoded for `categorical_crossentropy` training.
- **Train/Val Split** — 80% training / 20% validation, stratified by class.
- **Data Augmentation (training only)** — Random horizontal/vertical flips, rotation, zoom (10%), brightness adjustment ($\pm 20\%$). Applied via TensorFlow augmentation pipeline.

4. Model Architecture & Training

4.1 Scratch Model — CNN from Scratch

Custom 3-block CNN, no pretrained weights. Model file: `best_model.keras` (3.56 MB).

Layer / Block	Output Shape	Config	Parameters
Input	(64, 64, 3)	—	0
Conv Block 1 (x2 Conv2D)	(32, 32, 32)	32 filters, 3x3, BatchNorm, ReLU	~18,752
MaxPool2D + Dropout(0.25)	(32, 32, 32)	MaxPool 2x2	128
Conv Block 2 (x2 Conv2D)	(16, 16, 64)	64 filters, 3x3, BatchNorm, ReLU	~73,856
MaxPool2D + Dropout(0.25)	(16, 16, 64)	MaxPool 2x2	256
Conv Block 3 (x2 Conv2D)	(8, 8, 128)	128 filters, 3x3, BatchNorm, ReLU	~295,040
MaxPool2D + Dropout(0.40)	(8, 8, 128)	MaxPool 2x2	512
GlobalAveragePooling2D	(128,)	Replaces Flatten	0
Dense(256, ReLU) + Dropout(0.5)	(256,)	L2 regularization	~33,024
Dense(5, Softmax)	(5,)	Output — 5 classes	1,285
TOTAL	—	~322K (index.html stat)	~422,853 (sum of above)

4.2 Fine-Tuned Model — Two-Phase Transfer Learning

The Fine-Tune model uses a pretrained base trained in two phases. Model file: `plant_disease_model_zipped.keras` (19 MB).

Phase	Description	Trainable Params	Accuracy	Time
Phase 1: Feature Extraction	Base model frozen — only Dense classification head trained	166K	89.03%	7 min
Phase 2: Fine-Tuning (FINAL)	Base model unfrozen — full end-to-end fine-tuning at lower LR	2.42M	91.82%	4 min

Both phases use `ReduceLROnPlateau` learning rate scheduling and data augmentation via TensorFlow pipeline, as noted in the Fine-Tune UI.

4.3 Shared Training Techniques (both models, from UI feature list)

- Batch normalization and dropout regularization after convolutional and dense layers.
- Data augmentation with TensorFlow pipeline (flips, rotation, zoom, brightness).
- `ReduceLROnPlateau` learning rate scheduling.
- Real-time Sobel and Laplacian edge filter visualization via OpenCV (computed per inference request).
- Conv2D feature map visualization in the frontend UI.

5. Model Performance

All numbers below are taken directly from the deployed `index.html` pages of each Space.

Model	Accuracy	Trainable Params	Inference Speed	Training Time	Classes
Scratch (CNN from scratch)	89%	~322K	<2 seconds	12 min	5
Fine-Tune Phase 1 (frozen base)	89.03%	166K	25ms	7 min	15
Fine-Tune Phase 2 — FINAL	91.82%	2.42M	25ms	4 min	15

6. Deployment — Web Applications

Both models are deployed as Flask web applications in Docker containers on Hugging Face Spaces. All details below are from the deployed source files.

6.1 Shared Technology Stack (both apps — identical)

Component	Details
Base image	python:3.10-slim
System dependencies	libgl1, libglib2.0-0 (for OpenCV headless)
Backend framework	Flask + flask-cors
ML framework	tensorflow-cpu>=2.16.0, keras>=3.5.0
Image processing	opencv-python-headless
Numerics	numpy<2.0.0
Exposed port	7860 (Hugging Face Spaces standard)
Frontend	Vanilla HTML5 / CSS3 / JavaScript (Inter + Space Grotesk fonts, Font Awesome icons, Chart.js 4.4.0)
Frontend features	Drag-and-drop multi-image upload (JPG/PNG/JPEG, max 10MB), Filter Response toggle, Feature Maps toggle, Confidence Score toggle, Download All Results, progress bar during analysis

6.2 Scratch App specifics

- URL: https://huggingface.co/spaces/Khuzaima123/plant-disease-classifier_Scratch
- Model file: best_model.keras (3.56 MB)
- Classes: 5 — Healthy, Early Blight, Late Blight, Leaf Mold, Septoria Leaf Spot
- Footer link: redirects to the Fine-Tune model (plant-guard-ai space)

6.3 Fine-Tune App specifics

- URL: <https://huggingface.co/spaces/Khuzaima123/plant-guard-ai>
- Model file: plant_disease_model_zipped.keras (19 MB)
- Classes: 15 across Pepper Bell (2), Potato (3), Tomato (10)
- predict.py — included for local CLI inference with matplotlib bar chart visualization (color: green for healthy, red for disease)

6.4 API Endpoint (both apps — identical structure)

Endpoint	Method	Description
GET /	GET	Serves index.html frontend
POST /classify	POST	Accepts multipart/form-data image file. Returns JSON: { disease, confidence (%), probabilities (all class %), filters: { sobel (base64 JPEG), laplacian

Endpoint	Method	Description
		(base64 JPEG), featureMap (base64 JPEG — currently Laplacian placeholder) } }

6.5 Keras Compatibility Fix (both apps — identical code)

Both app.py files contain an identical monkey-patch to fix a Keras 3.x / saved model version mismatch. The Dense layer `__init__` is patched to silently drop the unrecognized `quantization_config` keyword argument before calling the original initializer, allowing `best_model.keras` and `plant_disease_model_zipped.keras` to load on `keras>=3.5.0` without errors.

7. Challenges & Limitations

7.1 Challenges

- Class imbalance — Unequal class sizes in the training data cause the model to bias toward majority classes without weighted loss.
- Resolution vs. compute trade-off — Smaller input sizes reduce compute but lose fine-grained lesion texture, hurting recall on visually similar diseases.
- Overfitting — Without augmentation, validation accuracy lagged training accuracy significantly. BatchNorm, Dropout, and augmentation were needed to close this gap.
- Keras version compatibility — A `quantization_config` mismatch between the saved .keras model and `keras>=3.5.0` required a runtime monkey-patch in both deployments.
- Feature map placeholder — The `/classify` endpoint currently returns the Laplacian filter image as `featureMap`; true intermediate CNN activations are not yet extracted server-side.

7.2 Limitations

- Closed-world assumption — Models only recognize disease classes seen in training. Novel or co-infections produce a wrong class label instead of an out-of-distribution signal.
- No spatial localization — Models output a class label without highlighting which leaf region drove the prediction. Users cannot visually verify the decision.
- Lab vs. field gap — PlantVillage images are under controlled lighting; real-world field images with shadows, occlusion, and mixed lighting can degrade performance.
- CPU-only inference — Both deployments use `tensorflow-cpu`; GPU inference would reduce the <2s latency further.

8. Suggested Improvements

- Grad-CAM — Generate saliency maps to show which leaf region activates the disease prediction, replacing the current Laplacian placeholder with meaningful spatial explanations.
- True feature map extraction — Extract and return actual intermediate Conv2D activations from the model server-side rather than using the Laplacian as a proxy.
- Higher input resolution — Increase input size to 128x128 or 224x224 for the Scratch model to capture finer lesion textures.
- Weighted loss — Use focal loss or class-weighted cross-entropy to handle class imbalance more robustly.
- OOD detection — Add a softmax threshold or secondary OOD head so the model abstains rather than misclassifying unknown inputs.
- Field image domain adaptation — Fine-tune on real field images to close the lab-to-field gap.
- Expand class coverage — Extend the Fine-Tune model toward full 38-class PlantVillage coverage across all 14 crop species.

9. Conclusion

PlantGuard AI delivers two production-deployed plant disease classification systems built on the PlantVillage dataset:

- Scratch Model: Custom 3-block CNN, no pretrained weights, 89% accuracy, ~322K parameters, 5 Tomato disease classes, <2s inference. Live at: huggingface.co/spaces/Khuzaima123/plant-disease-classifier_Scratch
- Fine-Tuned Model: Transfer-learned, 91.82% accuracy after two-phase fine-tuning, 2.42M trainable parameters, 15 classes across Pepper Bell / Potato / Tomato, 25ms inference. Live at: huggingface.co/spaces/Khuzaima123/plant-guard-ai

Both systems share an identical Flask + Docker deployment stack with real-time Sobel and Laplacian visualization, multi-image batch upload, and Chart.js-powered training analytics in the frontend. Future work will focus on Grad-CAM localization, true feature map extraction, higher-resolution inputs, and field-image domain adaptation.